

计算机科学与技术学院神经网络与深度学习课程实验报告

实验题目：全连接网络实现鸢尾花数据分类		学号：202300130183
日期：2025/9/18	班级：2023 级智能班	姓名：宋浩宇
Email: zhitian0420@gmail		
实验目的： 1.理解全连接神经网络（FCNN）的基本原理和结构。 2.掌握使用全连接神经网络对鸢尾花数据集进行分类的方法。 3.学习如何评估全连接神经网络的分类性能，并调整网络参数以优化性能。 4.理解验证集在模型调参中的作用，掌握通过交叉验证选择最优超参数的方法。		
实验软件和硬件环境： 软件环境： 主系统：Windows 11 家庭中文版 23H2 ; ArchLinux 编辑器：Visual Studio Code ; PyCharm 2025.1.2 Python 解释器：Python 3.9 Pytorch 版本：2.8.0 CUDA 版本：12.8.97 硬件环境： CPU：13th Gen Intel(R) Core(TM) i9-13980HX 2.20 GHz 内存：32.0 GB (31.6 GB 可用) 磁盘驱动器：NVMe WD_BLACKSN850X2000GB 显示适配器：NVIDIA GeForce RTX 4080 Laptop GPU CUDA 核心数：7424		
实验原理和方法： 实验原理为 FCNN 算法，通过人工测试来找合适的学习率、隐藏层数。		
实验步骤：（不要求罗列完整源代码） 首先导入、划分、标准化数据集 <pre># 下载/加载数据集 iris = load_iris() # 标准化数据 scaler = StandardScaler() iris.data = scaler.fit_transform(iris.data) # 划分训练集和测试集 x_train, x_test, y_train, y_test = train_test_split(iris.data, iris.target, test_size=0.2, random_state=11,stratify=iris.target)</pre> 这里将 stratify 设置为 iris.target，让数据集是分层随机抽样的。 然后构建 FCNN 模型 <pre>hidden_size1 = 64 hidden_size2 = 32</pre>		

```

learning_rate = 0.005
epochs = 100
batch_size = 32
test_epochs = 10
test_batch_size = 16
class FCNN(nn.Module):
    def __init__(self, input_size=4, hidden_size1=hidden_size1,
hidden_size2=hidden_size2, output_size=3):
        super(FCNN, self).__init__()
        self.fc1 = nn.Linear(input_size, hidden_size1)
        self.fc2 = nn.Linear(hidden_size1, hidden_size2)
        self.fc3 = nn.Linear(hidden_size2, output_size)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.2)
    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.dropout(x)
        x = self.fc2(x)
        x = self.relu(x)
        x = self.dropout(x)
        x = self.fc3(x)
        return x
model = FCNN()
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)

```

这里把隐藏层神经元个数设置为可以调整的超参数，使用两层隐藏层，丢弃 20% 的样本，用 ReLU 函数作为激活函数，交叉熵作为损失函数，使用 adam 优化器为优化器，同样把学习率作为可设置的超参数。

接下来就可以训练和测试了

```

def train_loader(x_train, y_train, batch_size=batch_size):
    x_train = torch.from_numpy(x_train).float()
    y_train = torch.from_numpy(y_train).long()
    dataset = torch.utils.data.TensorDataset(x_train, y_train)
    loader = torch.utils.data.DataLoader(dataset,
batch_size=batch_size, shuffle=True)
    return loader
def test_loader(x_test, y_test, batch_size=test_batch_size):
    x_test = torch.from_numpy(x_test).float()
    y_test = torch.from_numpy(y_test).long()
    dataset = torch.utils.data.TensorDataset(x_test, y_test)
    loader = torch.utils.data.DataLoader(dataset,
batch_size=batch_size, shuffle=True)
    return loader

```

```

if __name__ == '__main__':
    # print(x_train)
    # print(x_test)
    # print(y_train)
    # print(y_test)
    train_loader = train_loader(x_train, y_train)
    for epoch in range(epochs):
        for i, (inputs, labels) in enumerate(train_loader):
            optimizer.zero_grad()
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()
            # if (epoch+1) % 10 == 0:
            print('Epoch [{} / {}], Loss: {:.4f}'.format(epoch+1,
epochs, loss.item()))
        # 预测
        model.eval()
        test_correct = 0
        test_loader = test_loader(x_test, y_test)
        test_results = []
        with torch.no_grad():
            for test_epoch in range(test_epochs):
                for inputs, labels in test_loader:
                    outputs = model(inputs)
                    test_correct = 0
                    _, predicted = torch.max(outputs.data, 1)
                    test_correct += (predicted == labels).sum().item()
                    print(outputs)
                    print(predicted)
                    print(labels)
                    print("{} / {}".format(test_correct, len(labels)))
                    print('Epoch [{} / {}], Accuracy:
{:.8f}%'.format(test_epoch+1, test_epochs, 100 * test_correct /
len(labels)))
                test_results.append(100 * test_correct / len(labels))
            print('Test Accuracy: {:.8f}%'.format(np.mean(test_results)))

```

这个代码得到的最终结果是：

Epoch [10/10], Accuracy: 100.00000000%

Test Accuracy: 100.00000000%

结论分析与体会：

从结果上来看，对于 iris 这个数据集，使用 FCNN 方法进行预测，从预测准确率上来看，FCNN 能够较好的解决 iris 数据集上的预测这个问题，且成功率较高。并且上文中给出的超参数就是比较合适的超参数组成之一了。

就实验过程中遇到和出现的问题，你是如何解决和处理的，自拟 1—3 道问答题：

问题 1： 对于这些超参数没有很好的找最优或较优解的稳定方法。

解答 1： 控制变量一个一个改着试